02_matching_exacto_scvs

May 9, 2026

1 02. Matching Exacto con SCVS

1.1 Goal

Use SCVS company catalogs to recover exact RUC matches for both normalized company universes.

1.2 Inputs

- outputs/leads_companies_clean.csv
- outputs/horas_empresas_clean.csv
- 02_data_cleaning/data_super_compañias/

1.3 Outputs

- outputs/leads_ruc_exact.csv
- outputs/horas_ruc_exact.csv
- outputs/leads_ruc_sugerido_scvs.csv
- outputs/horas_ruc_sugerido_scvs.csv

[8]:
```
#   Helpers y rutas
import os
from pathlib import Path
import re, unicodedata
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore", category=UserWarning, module="openpyxl")

try:
    from IPython.display import display
except Exception:
    def display(x): print(x)

def find_project_root(start=None):
    start = (start or Path.cwd()).resolve()
    for p in [start, *start.parents]:
        if (p / "01_data_ingestion_enrichment").is_dir() and (p /␣
 ↪"02_data_cleaning").is_dir():
            return p
```

```
    raise FileNotFoundError(f"No se pudo localizar la raíz. cwd: {start}")

def ensure_dir(d): Path(d).mkdir(parents=True, exist_ok=True); return Path(d)

def save_df_csv(df, path, *, index=False, encoding="utf-8-sig"):
    path = Path(path); ensure_dir(path.parent)
    df.to_csv(path, index=index, encoding=encoding)
    print(f"[OK] Guardado: {path.resolve()}  shape: {df.shape}")
    return path

def read_csv_checked(path, **kwargs):
    path = Path(path)
    if not path.exists(): raise FileNotFoundError(f"No existe: {path.
 ↪resolve()}")
    return pd.read_csv(path, **kwargs)

ROOT      = find_project_root()
CLEAN_DIR = ROOT / "02_data_cleaning"
CLEAN_OUT = CLEAN_DIR / "outputs"

# Buscar data_super_compañias en ambas ubicaciones posibles
_SCVS_CANDIDATES = [CLEAN_DIR / "data_super_compañias", ROOT /␣
 ↪"01_data_ingestion_enrichment" / "data_super_compañias"]
SCVS_DIR = next((p for p in _SCVS_CANDIDATES if p.exists()), None)
if SCVS_DIR is None:
    raise FileNotFoundError(f"No encontré data_super_compañias en:␣
 ↪{_SCVS_CANDIDATES}")

print(f"[CONFIG] ROOT     : {ROOT}")
print(f"[CONFIG] CLEAN_OUT: {CLEAN_OUT}")
print(f"[CONFIG] SCVS_DIR : {SCVS_DIR}")
```

```
[CONFIG] ROOT     : E:\TESIS MAESTRIA\Desarrollo_clustering_maestria
[CONFIG] CLEAN_OUT: E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\outputs
[CONFIG] SCVS_DIR : E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\data_super_compañias
```

[9]:
```
#   Funciones de normalización y carga SCVS

def _strip_accents(s: str) -> str:
    return "".join(ch for ch in unicodedata.normalize("NFKD", s) if not␣
 ↪unicodedata.combining(ch))

def normalize_company_name(value) -> str:
    if value is None: return ""
    if isinstance(value, float) and np.isnan(value): return ""
```

```
    s = str(value).strip()
    if not s: return ""
    s = _strip_accents(s); s = s.upper()
    s = re.sub(r"[^A-Z0-9 ]+", " ", s)
    s = re.sub(r"\b(SA|S\s*A|S\.A\.?|S\.A\.S\.?|SAS|LTDA|CIA|C\.?IA\.?
 ↪|COMPANIA|COMPAÑIA|CORP|INC|LLC|C\.L\.?|C\.?LTDA\.?|\&|Y)\b", " ", s)
    s = re.sub(r"\b(DE|DEL|LA|EL|LOS|LAS)\b", " ", s)
    return re.sub(r"\s+", " ", s).strip()

def _normalize_ruc(value) -> str:
    if value is None: return ""
    if isinstance(value, float) and np.isnan(value): return ""
    s = re.sub(r"\D+", "", str(value).strip().replace(".0", ""))
    return s

def _norm_col(col: str) -> str:
    s = "" if col is None else str(col)
    s = s.strip().upper()
    s = "".join(c for c in unicodedata.normalize("NFKD", s) if not unicodedata.
 ↪combining(c))
    return re.sub(r"[^A-Z0-9]+", "", s)

def _pick_col(cols, patterns):
    cols_u = [str(c).upper() for c in cols]
    for pat in patterns:
        for i, cu in enumerate(cols_u):
            if pat in cu: return cols[i]
    return None

def _detect_excel_header_row(fp: Path, max_rows: int = 80) -> int:
    try: raw = pd.read_excel(fp, header=None, nrows=max_rows)
    except Exception: return 0
    best_i, best_score = 0, -1
    for i in range(len(raw)):
        vals = [v for v in raw.iloc[i].tolist() if str(v).lower() != "nan"]
        if not vals: continue
        row_text = " ".join(map(str, vals)).upper()
        score = (sum(tok in row_text for tok in ["RUC", "IDENTIFIC"]) * 5
                 + sum(tok in row_text for tok in ["NOMBRE", "RAZON", "RAZÓN"])␣
 ↪* 3)
        if score > best_score:
            best_score = score; best_i = i
    return int(best_i)

def _load_super_companies(folder: Path):
    if not folder.exists():
        raise FileNotFoundError(f"No existe la carpeta SCVS: {folder}")
```

```
    files = sorted([p for p in folder.iterdir() if p.suffix.lower() in {".
↪xlsx", ".xls", ".csv"}])
    if not files:
        raise FileNotFoundError(f"No se encontraron archivos en {folder}")
    frames = []
    for fp in files:
        try:
            if fp.suffix.lower() == ".csv":
                df = pd.read_csv(fp)
            else:
                df = pd.read_excel(fp, header=_detect_excel_header_row(fp))
        except Exception as e:
            print(f"[SCVS] Saltado {fp.name}: {e}"); continue
        if df is None or df.empty: continue
        df.columns = [str(c).strip() for c in df.columns]
        name_col = _pick_col(df.columns, ["RAZON", "RAZÓN", "NOMBRE", "DENOM",␣
↪"COMPAÑ", "EMPRESA"])
        ruc_col  = _pick_col(df.columns, ["RUC", "IDENTIFIC", "CEDULA"])
        if name_col is None:
            obj = [c for c in df.columns if str(df[c].dtype) in␣
↪("object","string")]
            name_col = obj[0] if obj else None
        if ruc_col is None:
            best, best_sc = None, -1.0
            for c in df.columns:
                s = df[c].map(_normalize_ruc); non_e = float((s != "").mean());␣
↪l13 = float((s.str.len() == 13).mean())
                sc = non_e + 3.0*l13
                if sc > best_sc: best_sc = sc; best = c
            ruc_col = best
        if name_col is None or ruc_col is None:
            print(f"[SCVS] Sin columnas detectables en {fp.name}"); continue
        tmp = pd.DataFrame({"super_source": fp.name, "super_name_raw":␣
↪df[name_col].astype("string"), "super_ruc_raw": df[ruc_col]})
        tmp["super_name_raw"] = tmp["super_name_raw"].fillna("").map(lambda x:␣
↪x.strip())
        tmp["super_ruc"] = tmp["super_ruc_raw"].map(_normalize_ruc)
        tmp = tmp[tmp["super_ruc"].astype("string").str.len() == 13].copy()
        tmp["super_name_norm"] = tmp["super_name_raw"].
↪map(normalize_company_name)
        tmp = tmp[tmp["super_name_norm"].str.contains(r"[A-Z]", regex=True,␣
↪na=False) & (tmp["super_name_norm"].str.len() >= 3)]
        if tmp.empty: continue
        frames.append(tmp[["super_source", "super_name_raw", "super_name_norm",␣
↪"super_ruc"]])
    if not frames:
```

```
        raise ValueError("No se pudo armar un catálogo SCVS válido.")
    super_all = pd.concat(frames, ignore_index=True)
    super_lookup = (super_all.groupby(["super_name_norm", "super_ruc"],␣
 ↪as_index=False).size()
                    .sort_values(["super_name_norm", "size"], ascending=[True,␣
 ↪False]))
    super_best = super_lookup.drop_duplicates(subset=["super_name_norm"],␣
 ↪keep="first")[["super_name_norm", "super_ruc"]]
    return super_all, super_best

super_all, super_best = _load_super_companies(SCVS_DIR)
print(f"[SCVS] Registros totales: {len(super_all)}")
print(f"[SCVS] Nombres distinct (lookup): {len(super_best)}")
```

```
[SCVS] Registros totales: 215291
[SCVS] Nombres distinct (lookup): 214446
```

1.4 Migrate Here From Source Notebook

- _normalize_ruc
- _pick_col
- _load_super_companies
- Exact RUC merge for leads.
- Exact RUC merge for horas.

1.4.1 Suggested Source Cells

- Code cells: 21 and 22

[10]:
```
#   Cargar universos normalizados desde staging
leads_clean = read_csv_checked(CLEAN_OUT / "leads_companies_clean.csv")
horas_clean  = read_csv_checked(CLEAN_OUT / "horas_empresas_clean.csv")

print(f"[LEADS]  {leads_clean.shape}  cols: {leads_clean.columns.tolist()}")
print(f"[HORAS]  {horas_clean.shape}  cols: {horas_clean.columns.tolist()}")

#   Match exacto LEADS  SCVS
leads_ruc_exact = (
    leads_clean.merge(super_best, left_on="Company_norm",␣
 ↪right_on="super_name_norm", how="left")
    .drop(columns=["super_name_norm"])
    .rename(columns={"super_ruc": "RUC"})
)
n_leads = len(leads_ruc_exact); n_leads_ok = int(leads_ruc_exact["RUC"].notna().
 ↪sum())
print(f"\n[LEADS] Total: {n_leads}  Con RUC exacto SCVS: {n_leads_ok}␣
 ↪({n_leads_ok/max(n_leads,1)*100:.1f}%)")
```

```
#   Match exacto HORAS  SCVS
horas_ruc_exact = (
    horas_clean.merge(super_best, left_on="EMPRESA_norm",␣
 ↪right_on="super_name_norm", how="left")
    .drop(columns=["super_name_norm"])
    .rename(columns={"super_ruc": "RUC"})
)
n_horas = len(horas_ruc_exact); n_horas_ok = int(horas_ruc_exact["RUC"].notna().
 ↪sum())
print(f"[HORAS] Total: {n_horas}  Con RUC exacto SCVS: {n_horas_ok}␣
 ↪({n_horas_ok/max(n_horas,1)*100:.1f}%)")

#   Sugerencias fuzzy SCVS (solo para los sin RUC)
UMBRAL_SCVS = int(os.getenv("SCVS_FUZZY_THRESHOLD", "80"))
choice_names = super_best["super_name_norm"].dropna().tolist()
ruc_by_name  = dict(zip(super_best["super_name_norm"], super_best["super_ruc"]))

def _fuzzy_scvs(unmatched: pd.DataFrame, q_col: str, raw_col: str) -> pd.
 ↪DataFrame:
    rows = []
    try:
        from rapidfuzz import process, fuzz
        for _, r in unmatched.iterrows():
            q = str(r.get(q_col, "") or "").strip()
            if not q: continue
            best = process.extractOne(q, choice_names, scorer=fuzz.
 ↪token_set_ratio)
            if best and float(best[1]) >= UMBRAL_SCVS:
                rows.append({"raw": r.get(raw_col, ""), "norm": q,␣
 ↪"best_scvs_norm": best[0], "score": float(best[1]), "RUC": ruc_by_name.
 ↪get(best[0], "")})
    except ImportError:
        from difflib import SequenceMatcher
        for _, r in unmatched.iterrows():
            q = str(r.get(q_col, "") or "").strip()
            if not q: continue
            best_name, best_sc = None, -1.0
            for cand in choice_names:
                sc = SequenceMatcher(None, q, cand).ratio() * 100
                if sc > best_sc: best_sc = sc; best_name = cand
            if best_name and best_sc >= UMBRAL_SCVS:
                rows.append({"raw": r.get(raw_col, ""), "norm": q,␣
 ↪"best_scvs_norm": best_name, "score": float(best_sc), "RUC": ruc_by_name.
 ↪get(best_name, "")})
    df = pd.DataFrame(rows) if rows else pd.DataFrame(columns=["raw", "norm",␣
 ↪"best_scvs_norm", "score", "RUC"])
```

```
    return df.sort_values("score", ascending=False).reset_index(drop=True) if␣
 ↪not df.empty else df

leads_ruc_sugerido_scvs = _fuzzy_scvs(leads_ruc_exact[leads_ruc_exact["RUC"].
 ↪isna()], "Company_norm", "Company_raw")
horas_ruc_sugerido_scvs = _fuzzy_scvs(horas_ruc_exact[horas_ruc_exact["RUC"].
 ↪isna()],  "EMPRESA_norm",  "EMPRESA_raw")

print(f"\n[LEADS] Sugerencias fuzzy SCVS (>={UMBRAL_SCVS}):␣
 ↪{len(leads_ruc_sugerido_scvs)}")
print(f"[HORAS] Sugerencias fuzzy SCVS (>={UMBRAL_SCVS}):␣
 ↪{len(horas_ruc_sugerido_scvs)}")

#   Guardar outputs
_ = save_df_csv(leads_ruc_exact,          CLEAN_OUT / "leads_ruc_exact.csv")
_ = save_df_csv(horas_ruc_exact,          CLEAN_OUT / "horas_ruc_exact.csv")
_ = save_df_csv(leads_ruc_sugerido_scvs,  CLEAN_OUT / "leads_ruc_sugerido_scvs.
 ↪csv")
_ = save_df_csv(horas_ruc_sugerido_scvs,  CLEAN_OUT / "horas_ruc_sugerido_scvs.
 ↪csv")

print("\n=== Vista previa leads_ruc_exact ===")
display(leads_ruc_exact.head(10))
```

```
[LEADS]  (326, 2)  cols: ['Company_raw', 'Company_norm']
[HORAS]  (119, 2)  cols: ['EMPRESA_raw', 'EMPRESA_norm']

[LEADS] Total: 326  Con RUC exacto SCVS: 6 (1.8%)
[HORAS] Total: 119  Con RUC exacto SCVS: 18 (15.1%)

[LEADS] Sugerencias fuzzy SCVS (>=80): 181
[HORAS] Sugerencias fuzzy SCVS (>=80): 72
[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning
\outputs\leads_ruc_exact.csv  shape: (326, 3)
[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning
\outputs\horas_ruc_exact.csv  shape: (119, 3)
[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning
\outputs\leads_ruc_sugerido_scvs.csv  shape: (181, 5)
[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning
\outputs\horas_ruc_sugerido_scvs.csv  shape: (72, 5)

=== Vista previa leads_ruc_exact ===
```

	Company_raw	Company_norm	RUC
0	7-ELEVEN MEXICO	7 ELEVEN MEXICO	NaN
1	ABBOTT	ABBOTT	NaN
2	Abbvie	ABBVIE	NaN
3	ACCO BRANDS	ACCO BRANDS	NaN

4	ACH FOOD COMPANIES, INC	ACH FOOD COMPANIES	NaN
5	ACTINVER	ACTINVER	NaN
6	ADAMANTINE	ADAMANTINE	NaN
7	AFP Genesis	AFP GENESIS	NaN
8	AIG	AIG	NaN
9	Akros	AKROS	1791148800001

[11]:
```
#   Verificación final
required_outputs = [
    CLEAN_OUT / "leads_ruc_exact.csv",
    CLEAN_OUT / "horas_ruc_exact.csv",
    CLEAN_OUT / "leads_ruc_sugerido_scvs.csv",
    CLEAN_OUT / "horas_ruc_sugerido_scvs.csv",
]
for p in required_outputs:
    if not p.exists():
        raise FileNotFoundError(f"Output faltante: {p}")
    df_chk = pd.read_csv(p)
    print(f"[OK] {p.name:45s}  shape={df_chk.shape}")
print("\n  Notebook 02_matching_exacto_scvs completado correctamente.")
```

```
[OK] leads_ruc_exact.csv                           shape=(326, 3)
[OK] horas_ruc_exact.csv                           shape=(119, 3)
[OK] leads_ruc_sugerido_scvs.csv                   shape=(181, 5)
[OK] horas_ruc_sugerido_scvs.csv                   shape=(72, 5)

  Notebook 02_matching_exacto_scvs completado correctamente.
```